

Moduldokumentation – DPP-Backend: APIController & zugehörige Klassen

Projekt: dpp-backend

Version: 0.0.1-SNAPSHOT

Sprache / Framework: Java 21 · Spring Boot 3.3.4 · MongoDB

Stand: Mai 2026

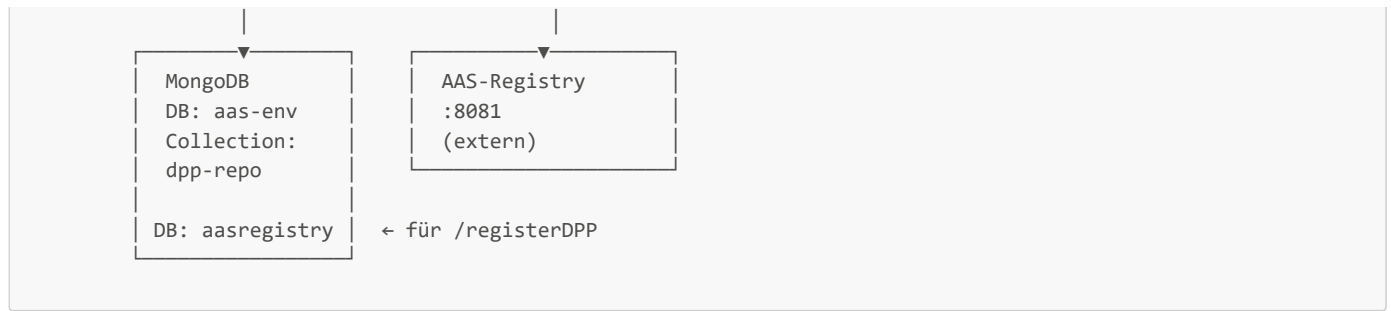
Inhaltsverzeichnis

1. Systemübersicht & Architektur
2. Konfiguration & Start
 - 2.1 DppBackendApplication
 - 2.2 MongoConfig
 - 2.3 application.yml
 - 2.4 pom.xml
3. Datenmodell – MongoDppTemplate
4. Repository-Interface – MongoDBInterface
5. Utility-Klassen
 - 5.1 Base64DPP
 - 5.2 ValidateDPP
 - 5.3 APIUtilsDPP
6. APIController – Endpunkte im Detail
 - 6.1 Klassenstruktur & Initialisierung
 - 6.2 GET /health
 - 6.3 POST /dpps
 - 6.4 GET /dpps/{dppId}
 - 6.5 GET /dpps/{dppId}/collections/{elementId}
 - 6.6 GET /dpps/{dppId}/elements/{elementPath}
 - 6.7 PATCH /dpps/{dppId}/collections/{elementId}
 - 6.8 PATCH /dpps/{dppId}/elements/{elementPath}
 - 6.9 GET /dppsByProductId/{productId}
 - 6.10 GET /dppsByProductIdAndDate/{productId}
 - 6.11 DELETE /dpps/{dppId}
 - 6.12 POST /dppsByProductIds
 - 6.13 PATCH /dpps/{dppId}
 - 6.14 POST /registerDPP
 - 6.15 Private Hilfsmethode `extractAndFilterSubmodels`
7. Fehlerbehandlung & Antwortformat
8. Bekannte Einschränkungen & Verbesserungspotenzial

1. Systemübersicht & Architektur

Das DPP-Backend ist ein **Spring-Boot-REST-Service**, der als Datenvermittler zwischen dem AAS-Ökosystem (Asset Administration Shell) und einer MongoDB-Datenbank fungiert. Es implementiert die Kernoperationen für **Digital Product Passports (DPP)**.





Datenbankschema (vereinfacht):

```

{
  "_id": "<shellId>",
  "dpps": [
    {
      "_id": "<dppId>",
      "productId": "<base64-kodiert>",
      "createdAt": "<Unix-Millisekunden>",
      "version": "1.0",
      "submodels": [
        { "reference": "<submodel-URI>", "name": "NamePlate", "version": "1.0" }
      ]
    }
  ]
}

```

Ein Shell-Dokument kann **mehrere DPPs** in einem Array enthalten. Der **shellId** ist der **_id**-Schlüssel des Dokuments, der **dppId** identifiziert den einzelnen DPP-Eintrag innerhalb des Arrays.

2. Konfiguration & Start

2.1 DppBackendApplication

Datei: `src/main/java/com/dpp/DppBackendApplication.java`

```

@SpringBootApplication
public class DppBackendApplication {

    public static void main(String[] args) {
        SpringApplication.run(DppBackendApplication.class, args);
    }

    @Bean
    public CommandLineRunner run(MongoDBInterface mongoInterface, MongoTemplate mongoTemplate) {
        return args -> {
            System.out.println("DB Name: " + mongoTemplate.getDb().getName());

            if (!mongoTemplate.collectionExists("dpp-repo")) {
                mongoTemplate.createCollection("dpp-repo");
            }
        };
    }
}

```

Erklärung:

- `@SpringBootApplication` aktiviert Auto-Configuration, Component-Scan und Konfigurationsunterstützung in einem.
- Der `CommandLineRunner`-Bean wird **beim Start** ausgeführt. Er prüft, ob die MongoDB-Collection `dpp-repo` existiert, und legt sie andernfalls an. Dies verhindert Fehler beim ersten Hochfahren gegen eine leere Datenbank.
- Der Bean erhält `MongoDBInterface` und `MongoTemplate` per Dependency Injection – beide werden von Spring Data MongoDB automatisch bereitgestellt.

2.2 MongoConfig

Datei: `src/main/java/com/dpp/MongoConfig.java`

```
@Configuration
public class MongoConfig extends AbstractMongoClientConfiguration {

    @Override
    protected String getDatabaseName() {
        return "aas-env";
    }

    @Override
    public MongoClient mongoClient() {
        String mongoUri = System.getenv("SPRING_DATA_MONGODB_URI");
        if (mongoUri == null || mongoUri.isEmpty()) {
            mongoUri = "mongodb://mongoAdmin:mongoPassword@127.0.0.1:27017/aas-env?authSource=admin";
        }

        ConnectionString connectionString = new ConnectionString(mongoUri);
        MongoClientSettings settings = MongoClientSettings.builder()
            .applyConnectionString(connectionString)
            .build();
        return MongoClient.create(settings);
    }
}
```

Erklärung:

- Erweitert `AbstractMongoClientConfiguration`, um Spring Data MongoDB vollständig zu konfigurieren.
- `getDatabaseName()` legt die primäre Datenbank auf `aas-env` fest.
- Die Verbindungs-URI wird aus der **Umgebungsvariable** `SPRING_DATA_MONGODB_URI` gelesen. Falls diese nicht gesetzt ist, wird ein lokaler Fallback-Wert verwendet (nützlich für lokale Entwicklung).
- `MongoClientSettings` ermöglicht zukünftige Erweiterungen (TLS, Connection-Pooling etc.).

2.3 application.yml

```
server:
  port: 8080

management:
  endpoints:
    web:
      exposure:
        include: health,info
  endpoint:
    health:
      show-details: always
```

Erklärung:

- Der Server läuft auf Port `8080`.
- Spring Actuator stellt die Endpunkte `/actuator/health` und `/actuator/info` bereit. `show-details: always` gibt detaillierte Statusinformationen zurück (DB-Verbindungsstatus etc.).

2.4 pom.xml – Abhängigkeiten

Dependency	Zweck
<code>spring-boot-starter-web</code>	REST-Endpunkte, eingebetteter Tomcat

Dependency	Zweck
spring-boot-starter-data-mongodb	MongoDB-Integration, <code>MongoTemplate</code> , Repository-Support
spring-boot-starter-webflux	Stellt <code>RestClient</code> bereit (synchrones HTTP-Clienting gegen AAS-Registry)
spring-boot-starter-actuator	Health- und Info-Endpunkte

Hinweis: `spring-boot-starter-webflux` wird ausschließlich für die `RestClient`-Klasse eingebunden. Das Backend selbst ist **nicht reaktiv** – alle Operationen sind synchron/blockierend.

3. Datenmodell – `MongoDppTemplate`

Datei: `src/main/java/com/dpp/MongoDppTemplate.java`

```
@Document(collection = "dpp-repo")
public class MongoDppTemplate {

    @Id
    @Field("dppId")
    @JsonProperty("dppId")
    private String dppId;

    @JsonProperty("productId")
    private String productId;

    @JsonProperty("createdAt")
    private String createdAt;

    @JsonProperty("version")
    private String version;

    @JsonProperty("submodels")
    private List<Submodels> submodels = new ArrayList<>();

    // Getter & Setter ...

    public static class Submodels {
        private String reference; // Die URI des Submodells (z. B. AAS-Identifizier)
        private String version;
        private String name;      // Normierter Name (z. B. "NamePlate", "Circularity")

        public Submodels() {}
        public Submodels(String path, String name, String version) { ... }
        // Getter & Setter ...
    }
}
```

Erklärung:

- `@Document(collection = "dpp-repo")` bindet die Klasse an die MongoDB-Collection `dpp-repo`.
- `@Id` markiert `dppId` als primären Schlüssel (`_id` in MongoDB). Das Feld wird durch `@Field("dppId")` explizit benannt, damit der JSON-Name `dppId` und der MongoDB-Feldname übereinstimmen.
- `createdAt` ist ein Unix-Millisekunden-Timestamp als String (kein `Date`-Objekt), was Zeitzonenprobleme vermeidet.
- `productId` wird **immer Base64-URL-kodiert** gespeichert (sichergestellt durch `Base64DPP.ensureEncoding()`).
- Die innere Klasse `Submodels` speichert referenzierte AAS-Submodelle mit URI, semantischem Namen und Version.

4. Repository-Interface – `MongoDBInterface`

Datei: `src/main/java/com/dpp/MongoDBInterface.java`

```
public interface MongoDBInterface extends MongoRepository<MongoDppTemplate, String> {
```

```
@Query("{ '_id': ?0 }")
@update("{ '$push': { 'dpps': ?1 } }")
void appendDppToShell(String shellId, MongoDppTemplate dppContent);
}
```

Erklärung:

- Erbt alle Standard-CRUD-Operationen von `MongoRepository`.
- Die Methode `appendDppToShell` nutzt Spring Data `@Query` + `@Update`, um ein `$push`-Update direkt als annotierte Methode zu formulieren – ohne manuelles `MongoTemplate`-Scripting.
- In der aktuellen Implementierung wird dieses Interface primär im `CommandLineRunner` für `count()` genutzt. Die eigentlichen DPP-Operationen laufen über `MongoTemplate`, da dieses flexiblere Aggregations- und Upsert-Operationen erlaubt.

5. Utility-Klassen

5.1 Base64DPP

Datei: `src/main/java/com/dpp/util/Base64DPP.java`

Diese Klasse stellt sicher, dass Identifier (insbesondere `productId` und Submodel-Referenzen) konsistent als **URL-sicheres Base64 ohne Padding** kodiert werden – eine Anforderung der AAS-API-Spezifikation.

`encodeIdentifier(String url)`

```
public static String encodeIdentifier(String url) {
    return Base64.getUrlEncoder().withoutPadding()
        .encodeToString(url.getBytes(StandardCharsets.UTF_8));
}
```

Kodiert einen beliebigen String in URL-sicheres Base64 (kein `+/=`, kein `=`-Padding). Wird direkt aufgerufen, wenn sicher ist, dass der Input noch unkodiert ist.

`decodeIdentifier(String identifier)`

```
public static String decodeIdentifier(String identifier) {
    try {
        return new String(Base64.getUrlDecoder().decode(identifier), StandardCharsets.UTF_8);
    } catch (IllegalArgumentException e) {
        return null; // Kein gültiger Base64-String
    }
}
```

Dekodiert einen Base64-Identifier zurück in den Klartext. Gibt `null` zurück, wenn der Input kein gültiges Base64 ist.

`ensureEncoding(String input)`

```
public static String ensureEncoding(String input) {
    if (input == null || input.isEmpty()) return input;
    if (isBase64Encoded(input)) return input; // Bereits kodiert → unverändert
    return encodeIdentifier(input); // Noch nicht kodiert → jetzt kodieren
}
```

Wichtigste Methode der Klasse. Prüft mittels `isBase64Encoded()`, ob der String bereits Base64 ist, und kodiert ihn nur dann, wenn nötig. Verhindert Doppelkodierung.

`isBase64Encoded(String input)` (*privat*)

```
private static boolean isBase64Encoded(String input) {
    try {
        Base64.getUrlDecoder().decode(input);
        return true;
    } catch (IllegalArgumentException e) {
        return false;
    }
}
```

⚠ **Bekannte Schwäche:** Diese Methode versucht, den String zu dekodieren. Kurze alphanumerische Strings (z. B. "v1" oder "abc") können als gültiges Base64 interpretiert werden, auch wenn sie es semantisch nicht sind. In der Praxis treten dadurch keine Fehler auf, da Produkt-IDs typischerweise Sonderzeichen wie `:` oder `/` enthalten, die kein gültiges URL-Base64 ergeben.

5.2 ValidateDPP

Datei: `src/main/java/com/dpp/util/ValidateDPP.java`

```
public class ValidateDPP {

    public static boolean validateJsonTillFirstEntry(JsonNode dpp) {

        // 1. Prüfe Pflichtfelder auf oberster Ebene
        if (!dpp.has("shell") || !dpp.get("shell").has("id")) {
            return false;
        }

        JsonNode dppsArray = dpp.get("shell").path("dpps");

        // 2. dpps-Array muss existieren und mindestens einen Eintrag haben
        if (!dppsArray.isArray() || dppsArray.isEmpty()) {
            return false;
        }

        // 3. Pflichtfelder des ersten DPP-Eintrags prüfen
        JsonNode firstDpp = dppsArray.get(0);
        for (String field : new String[]{"productId", "version"}) {
            if (!firstDpp.has(field)) return false;
        }

        return true;
    }
}
```

Erklärung:

Validiert die Mindeststruktur eines eingehenden DPP-JSON-Payloads. Das erwartete Format ist:

```
{
  "shell": {
    "id": "<shellId>",
    "dpps": [
      {
        "productId": "...",
        "version": "..."
      }
    ]
  }
}
```

Die Methode gibt `false` zurück, wenn:

- kein `shell`-Objekt vorhanden ist,
- `shell.id` fehlt,
- das `dpps`-Array fehlt oder leer ist,
- `productId` oder `version` im ersten Eintrag fehlen.

5.3 APIUtilsDPP

Datei: `src/main/java/com/dpp/util/APIUtilsDPP.java`

Diese Klasse bündelt alle wiederverwendbaren Hilfsfunktionen des Controllers: MongoDB-Abfragen, AAS-API-Aufrufe und Antwort-Formatierung.

`getDppById(String dppId, MongoTemplate mongoTemplate)`

```
public static MongoDppTemplate getDppById(String dppId, MongoTemplate mongoTemplate) {
    Aggregation aggregation = Aggregation.newAggregation(
        Aggregation.match(Criteria.where("dpps._id").is(dppId)),
        Aggregation.unwind("dpps"),
        Aggregation.match(Criteria.where("dpps._id").is(dppId)),
        Aggregation.replaceRoot("dpps")
    );

    List<org.bson.Document> results = mongoTemplate
        .aggregate(aggregation, "dpp-repo", org.bson.Document.class)
        .getMappedResults();

    if (results.isEmpty()) return null;

    return mongoTemplate.getConverter().read(MongoDppTemplate.class, results.get(0));
}
```

Aggregations-Pipeline-Erklärung:

Stage	Beschreibung
<code>match</code> (1)	Findet das Shell-Dokument, das im <code>dpps</code> -Array einen Eintrag mit der gesuchten <code>_id</code> enthält
<code>unwind</code>	Entfaltet das <code>dpps</code> -Array – jeder Eintrag wird zu einem eigenen Dokument
<code>match</code> (2)	Filtert auf den exakten Eintrag mit der gesuchten <code>dppId</code> (nach dem Entfalten)
<code>replaceRoot</code>	Ersetzt das Root-Dokument durch den <code>dpps</code> -Eintrag selbst

Das Ergebnis ist ein einzelnes BSON-Dokument, das dann mittels `mongoTemplate.getConverter().read()` in ein `MongoDppTemplate`-Objekt deserialisiert wird.

`getDppByAggregatoin(Aggregation, MongoTemplate)` *(Typo im Namen)*

Generische Variante von `getDppById`, die eine bereits fertig konstruierte Aggregations-Pipeline entgegennimmt. Wird in `readElementCollection` verwendet.

`collectSubmodelData(ObjectMapper, MongoDppTemplate, RestClient, Logger)`

```
public static ObjectNode collectSubmodelData(...) {
    ObjectNode collectedSubmodels = mapper.createObjectNode();

    for (Submodels submodel : dpp.getSubmodels()) {
        String externalUrl = externalApiBase + "/submodels/"
            + Base64DPP.encodeIdentifier(submodel.getReference())
            + "/submodel-elements";

        JsonNode externalPayload = restClient.get()
            .uri(externalUrl).retrieve().body(JsonNode.class);
    }
}
```

```

        if (externalPayload != null && externalPayload.has("result")) {
            collectedSubmodels.putPOJO(submodel.getName(), externalPayload.get("result"));
        }
    }

    return collectedSubmodels;
}

```

Iteriert über alle Submodelle eines DPPs und ruft für jedes den AAS-Server unter `/submodels/{submodelId}/submodel-elements` ab. Die Ergebnisse werden unter dem jeweiligen Submodell-Namen (`"NamePlate"`, `"Circularity"` etc.) im zurückgegebenen `ObjectNode` zusammengeführt.

collectAssetInformation(ObjectMapper, MongoDppTemplate, RestClient, Logger)

Ruft `/shells/{productId}` auf dem AAS-Server ab und extrahiert das `assetInformation`-Feld aus der Antwort.

collectAdministration(ObjectMapper, MongoDppTemplate, RestClient, Logger)

Ruft ebenfalls `/shells/{productId}` ab und extrahiert das `administration`-Feld. Beide Methoden (`collectAssetInformation` und `collectAdministration`) machen identische HTTP-Aufrufe – hier besteht Optimierungspotenzial durch eine gemeinsame Methode.

create_generic_response(int code, String text, String messageType, ObjectMapper mapper)

```

public static ResponseEntity<ObjectNode> create_generic_response(
    int code, String text, String messageType, ObjectMapper mapper) {

    ObjectNode response = mapper.createObjectNode();
    ObjectNode inner = mapper.createObjectNode();
    ArrayNode arr = response.putArray("message");

    inner.put("code", code);
    inner.put("messageType", messageType);
    inner.put("correlationId", getStatusCode(code)); // Normierter Statusname
    inner.put("text", text);
    inner.put("timeStamp", Instant.now().toString());

    arr.add(inner);
    return ResponseEntity.status(code).body(response);
}

```

Erstellt eine normierte Fehlerantwort gemäß **prEN 18222:2025, Tabelle 16**. Der `correlationId`-Wert wird aus der internen `ERROR_MAP` aufgelöst:

HTTP-Code	correlationId
200	Success
201	SuccessCreated
400	ClientErrorBadRequest
404	ClientErrorResourceNotFound
500	ServerInternalError

extractAndFilterSubmodels(JsonNode resultsArray) (statische Methode)

```

public static List<MongoDppTemplate.Submodels> extractAndFilterSubmodels(JsonNode resultsArray) {
    Map<String, String> submodelMap = new HashMap<>();
    submodelMap.put("nameplate", "NamePlate");
    submodelMap.put("circularity", "Circularity");
    submodelMap.put("carbonfootprint", "CarbonFootPrint");
    submodelMap.put("handoverdocumentation", "HandoverDocumentation");
    submodelMap.put("technicaldata", "TechnicalData");
}

```



```

submodelMap.put("productcondition", "ProductCondition");
submodelMap.put("materialcomposition", "MaterialComposition");

for (JsonNode entry : resultsArray) {
    String fullPath = entry.path("keys").get(0).path("value").asText();

    // Normalisierung: Kleinbuchstaben, Sonderzeichen entfernen
    String normalizedPath = fullPath.toLowerCase()
        .replace("_", "").replace("-", "").replace(".", "").replace("/", "");

    for (Map.Entry<String, String> target : submodelMap.entrySet()) {
        if (normalizedPath.contains(target.getKey())) {
            list.add(new Submodels(fullPath, target.getValue(), "1.0"));
            break;
        }
    }
}
return list;
}

```

Parst die Antwort der AAS-Registry (/shells/{id}/submodel-refs) und filtert Submodelle nach einer Whitelist bekannter Submodell-Typen heraus. Der Pfad wird normalisiert (Kleinbuchstaben, keine Sonderzeichen), um robuste Substring-Matches zu ermöglichen.

6. APIController – Endpunkte im Detail

Datei: src/main/java/com/dpp/api/APIController.java

6.1 Klassenstruktur & Initialisierung

```

@RestController
@RequestMapping
public class APIController {

    private static final Logger logger = LoggerFactory.getLogger(APIController.class);

    private final ObjectMapper mapper;
    private final RestClient restClient = RestClient.create();
    private final MongoTemplate mongoTemplate; // Primäre DB (aas-env)
    private MongoTemplate aasRegistryTemplate; // Sekundäre DB (aasregistry)

    public APIController(ObjectMapper mapper, MongoTemplate mongoTemplate) {
        this.mapper = mapper;
        this.mongoTemplate = mongoTemplate;

        // Manuelle Verbindung zur zweiten Datenbank (aasregistry)
        String mongoUri = System.getenv("SPRING_DATA_MONGODB_URI");
        MongoClient manualClient = MongoClient.create(mongoUri);
        this.aasRegistryTemplate = new MongoTemplate(manualClient, "aasregistry");
    }
}

```

Erklärung:

- `@RestController` kombiniert `@Controller` und `@ResponseBody` – alle Methoden geben direkt HTTP-Responses zurück.
- `@RequestMapping` ohne Pfad registriert die Klasse am Root-Pfad. Jede Methode definiert ihren eigenen Pfad.
- `mongoTemplate` wird per Constructor Injection eingebunden (empfohlen gegenüber `@Autowired`).
- `aasRegistryTemplate` zeigt auf die **separate Datenbank** `aasregistry` und wird nur für `/registerDPP` verwendet. Die Verbindung wird manuell im Konstruktor aufgebaut, da Spring nur eine primäre Datenbank automatisch konfiguriert.
- `restClient` ist der synchrone HTTP-Client für Calls an den externen AAS-Server (Port 8081).

6.2 GET /health

```
@GetMapping("/health")
public ResponseEntity<ObjectNode> health() {
    ObjectNode node = mapper.createObjectNode();
    node.put("status", "UP");
    return ResponseEntity.ok(node);
}
```

Zweck: Einfacher Liveness-Check. Gibt {"status": "UP"} zurück.

Antwort: 200 OK

```
{ "status": "UP" }
```

6.3 POST /dpps

Zweck: Erstellt oder ergänzt einen Digital Product Passport in der primären Datenbank (aas-env).

Request-Body:

```
{
  "shell": {
    "id": "<shellId>",
    "dpps": [{
      "productId": "urn:example:product:123",
      "version": "1.0"
    }]
  }
}
```

Ablauf:

1. ValidateDPP.validateJsonTillFirstEntry(dpp)
 - └ false → 400 Bad Request
2. shellId = dpp.shell.id
 - dppEntry = dpp.shell.dpps[0]
3. dppId = Base64(productId + currentTimeMillis)
4. AAS-Call: GET {EXTERNAL_AAS_API_URL}/shells/{aasIdentifizier}/submodel-refs
 - └ Erfolg → filteredSubmodels via APIUtilsDPP.extractAndFilterSubmodels()
 - └ Fehler → filteredSubmodels = [] (Fehler wird nur geloggt, kein Abbruch)
5. dppContent = MongoDppTemplate(dppEntry)
 - dppContent.createdAt = currentTimeMillis
 - dppContent.dppId = dppId
 - dppContent.submodels = filteredSubmodels
 - dppContent.productId = Base64(productId)
6. MongoDB UPSERT auf Collection "dpp-repo":
 - query = { _id: shellId }
 - update = { \$push: { dpps: dppContent } }
 - Dokument wird angelegt (insert) oder der dpps-Array ergänzt (update)
7. Antwort: 201 Created { "status": "success", "dppId": "..." }

Code-Kernstück – Upsert:

```
Query query = new Query(Criteria.where("_id").is(shellId));
Update update = new Update().push("dpps", dppContent);
mongoTemplate.upsert(query, update, "dpp-repo");
```

`upsert()` ist atomar: Existiert das Dokument, wird `$push` ausgeführt; existiert es nicht, wird ein neues Dokument mit dem `dpps`-Array angelegt.

6.4 GET /dpps/{dppId}

Zweck: Liest einen einzelnen DPP anhand seiner ID und reichert die Antwort mit Daten vom AAS-Server an.

Pfadvariable: `dppId` – der Base64-kodierte DPP-Identifizier

Ablauf:

1. MongoDB-Aggregation (via `APIUtilsDPP-Pattern`):
 - `match` → `dpps._id == dppId`
 - `unwind` → `dpps`-Array entfalten
 - `match` → erneut auf `dppId` filtern
 - `replaceRoot` → `dpps`-Element als Root
2. `results.isEmpty()` → 404 Not Found
3. `dpp = mongoTemplate.getConverter().read(MongoDppTemplate, results[0])`
4. Parallel (sequenziell):
 - `collectSubmodelData()` → Submodell-Inhalte von AAS holen
 - `collectAssetInformation()` → `assetInformation` vom AAS-Shell
 - `collectAdministration()` → `administration` vom AAS-Shell
5. Antwort: 200 OK

Antwortstruktur:

```
{
  "status": "success",
  "dpp": { ... },
  "assetInformation": { "assetInformation": { ... } },
  "administration": { "administration": { ... } },
  "submodels_values": {
    "NamePlate": [ ... ],
    "Circularity": [ ... ]
  }
}
```

6.5 GET /dpps/{dppId}/collections/{elementId}

Zweck: Ruft alle Submodell-Referenzen einer bestimmten Collection (identifiziert durch `elementId`) für einen DPP ab.

Pfadvariablen:

- `dppId` – DPP-Identifizier
- `elementId` – Submodell-Referenz-ID (wird ggf. zu Base64 kodiert)

Ablauf:

1. `elementId = Base64DPP.ensureEncoding(elementId)`
2. DPP via Aggregation laden (`getDppByAggregatoin`)

```

3. Für jedes submodel in dpp.submodels:
    submodelBase64 = Base64DPP.ensureEncoding(submodel.reference)

    IF submodelBase64 == elementId:
        GET {AAS_URL}/shells/{submodelBase64}/submodel-refs
        → Antwort.result zurückgeben

4. Kein Match → 500 (sollte 404 sein)

```

6.6 GET /dpps/{dppId}/elements/{elementPath}

Zweck: Liest ein einzelnes Submodell-Element anhand eines Pfads der Form `<SubmodellName>.<ElementPfad>`.

Beispiel: `elementPath = "NamePlate.ManufacturerName"` → Sucht das Submodell `NamePlate` und ruft das Element `NamePlate.ManufacturerName` beim AAS-Server ab.

Ablauf:

```

1. submodel_name = elementPath.split(".")[0] // z. B. "NamePlate"

2. readDppById(dppId) aufrufen → dppTemplate extrahieren
   (POJONode-Cast aus dem Response-Body)

3. dppTemplate.submodels durchsuchen:
   submodel.name.toLowerCase() == submodel_name.toLowerCase()
   → submodel_identifizier = submodel.reference

4. submodel_identifizier = Base64DPP.ensureEncoding(submodel_identifizier)

5. GET {AAS_URL}/submodels/{submodel_identifizier}/submodel-elements/{elementPath}

6. Antwort: 200 OK { "status": "success", "payload": { ... } }

```

6.7 PATCH /dpps/{dppId}/collections/{elementId}

Zweck: Aktualisiert eine Collection (Submodell) eines DPPs über die AAS-Registry.

Ablauf:

```

1. elementId = Base64DPP.ensureEncoding(elementId)

2. DPP per getDppById laden

3. Submodell-Match suchen (wie in 6.5)

4. Bei Match: PATCH {AAS_URL}/shells/{submodelBase64}/submodel-refs
   mit übergebenem body

5. Antwort: 200 OK { "status": "success", "payload": <body> }

```

6.8 PATCH /dpps/{dppId}/elements/{elementPath}

Zweck: Aktualisiert einen einzelnen Feldwert innerhalb eines DPP-Dokuments direkt in MongoDB.

Eingabe-Validierung:

```

if (dppId == null || dppId.trim().isEmpty()) → 400 Bad Request
if (elementPath == null || elementPath.isEmpty()) → 400 Bad Request

```

```
if (updateValue == null) → 400 Bad Request
```

MongoDB-Update-Logik:

```
// Pfad-Konvention: "dpps.0.{elementPath}"
// Beispiel: elementPath = "version" → MongoDB-Pfad = "dpps.0.version"
String mongoPath = "dpps.0." + elementPath;

Query query = new Query(Criteria.where("dpps._id").is(dppId));
Update update = new Update().set(mongoPath, updateValue);
UpdateResult result = mongoTemplate.updateFirst(query, update, "dpp-repo");
```

⚠ **Bekannte Einschränkung:** "dpps.0." adressiert immer den **ersten** Eintrag im Array, unabhängig von der `dppId`. Bei mehreren DPPs in einem Shell-Dokument kann dies den falschen Eintrag treffen. Korrekt wäre ein Array-Filter: "dpps.\$[elem].version" mit `arrayFilter`.

6.9 GET /dppsByProductId/{productId}

Zweck: Sucht den **neuesten** DPP für eine gegebene `productId`.

Besonderheit: Bei mehreren Treffern wird `results.get(results.size() - 1)` verwendet – also der **letzte** Treffer in der Aggregationsreihenfolge. Da keine explizite Sortierung vorliegt, entspricht dies der natürlichen Dokumentreihenfolge in MongoDB.

6.10 GET /dppsByProductIdAndDate/{productId}

Zweck: Sucht einen DPP nach `productId` **und** einem exakten `createdAt`-Timestamp.

Query-Parameter: `?date=<timestamp>`

Aggregation-Kriterium:

```
Criteria.where("dpps.productId").is(productId)
    .and("dpps.createdAt").is(date)
```

Hinweis: Der Timestamp ist ein Unix-Millisekunden-String. Der Client muss den exakten Wert kennen. Er kann diesen über den Call `GET /dppsByProductId/{productId}` bekommen.

6.11 DELETE /dpps/{dppId}

Zweck: Entfernt einen einzelnen DPP-Eintrag aus dem `dpps`-Array.

```
Query query = new Query(Criteria.where("dpps._id").is(dppId));
Update update = new Update().pull("dpps", new org.bson.Document("_id", dppId));
UpdateResult result = mongoTemplate.updateFirst(query, update, "dpp-repo");
```

`$pull` entfernt alle Elemente aus dem Array, die der Bedingung entsprechen. Das umschließende Shell-Dokument bleibt erhalten (auch wenn `dpps` danach leer ist).

Antworten:

- 404 wenn kein Eintrag gelöscht wurde (`modifiedCount == 0`)
- 200 bei Erfolg

6.12 POST /dppsByProductIds

Zweck: Batch-Abfrage – liefert DPP-IDs für eine Liste von Produkt-IDs.

Request-Body: ["productId1", "productId2", ...]

Aggregation:

```
Aggregation.match(Criteria.where("dpps.productId").in(productIds)),
Aggregation.unwind("dpps"),
Aggregation.match(Criteria.where("dpps.productId").in(productIds)),
Aggregation.project()
  .and("dpps.productId").as("productId")
  .and("dpps._id").as("dppId")
```

Ergebnis: Gruppirt nach **productId** – jede Produkt-ID enthält eine Liste aller zugehörigen **dppIds**.

Antwortstruktur:

```
{
  "status": "success",
  "results": {
    "productId1": [{ "dppId": "..."}, { "dppId": "..."}],
    "productId2": [{ "dppId": "..."}]
  }
}
```

6.13 PATCH /dpps/{dppId}

Zweck: Aktualisiert einen DPP, indem der alte Eintrag gelöscht und ein neuer mit frischem Timestamp und neuer **dppId** eingefügt wird (Versionsfortschreibung).

Ablauf:

1. Alten DPP per Aggregation finden → productId und shellId extrahieren
2. Neuen DPP (newDpp) aufbauen:
 - newDppId = productId + currentTimeMillis
 - version = updateData.version ?? oldDpp.version
 - submodels = updateData.submodels ?? null
3. \$pull → alten DPP aus dpps-Array entfernen
4. \$push → neuen DPP einfügen
5. Antwort: 200 OK { "status": "success", "newDppId": "..."} }

Hinweis: Pull und Push sind **zwei separate** Datenbankoperationen, keine atomare Transaktion. Bei einem Fehler zwischen den beiden Schritten könnte der alte DPP bereits gelöscht sein, ohne dass der neue eingefügt wurde. Lösungsmöglichkeiten: Wir ändern die Konfiguration der MongoDB, was jedoch zu Konflikten mit den gesamten BaSyx Diensten führt, deshalb wird dieses Problem an dieser Stelle akzeptiert.

6.14 POST /registerDPP

Zweck: Wie **POST /dpps**, schreibt aber in die **sekundäre Datenbank aasregistry**.

```
// Primäre DB:
mongoTemplate.upsert(query, update, "dpp-repo");

// Sekundäre DB (nur hier):
aasRegistryTemplate.upsert(query, update, "dpp-repo");
```

Die interne Hilfsmethode `extractAndFilterSubmodels()` des Controllers verwendet eine leicht abweichende Whitelist (z. B. `"digitalnameplate"` statt `"nameplate"`).

6.15 Private Methode `extractAndFilterSubmodels` (im Controller)

```
private List<MongoDppTemplate.Submodels> extractAndFilterSubmodels(JsonNode resultsArray) {
    Map<String, String> submodelMap = new HashMap<>();
    submodelMap.put("digitalnameplate", "DigitalNamePlate");
    submodelMap.put("circularity", "Circularity");
    // ...

    for (JsonNode entry : resultsArray) {
        String fullPath = entry.path("keys").get(0).path("value").asText();
        String normalizedPath = fullPath.toLowerCase()
            .replace("_", "").replace("-", "").replace(".", "").replace("/", "");

        for (Map.Entry<String, String> target : submodelMap.entrySet()) {
            if (normalizedPath.contains(target.getKey())) {
                list.add(new Submodels(fullPath, target.getValue(), "1.0"));
                break;
            }
        }
    }
    return list;
}
```

Identisch mit `APIUtilsDPP.extractAndFilterSubmodels()`, jedoch mit abweichendem Submodell-Mapping (`"digitalnameplate"` vs. `"nameplate"`). Diese Duplikation sollte konsolidiert werden.

7. Fehlerbehandlung & Antwortformat

Erfolgsantwort (Standard)

```
{
  "status": "success",
  "dppId": "...",
  ...
}
```

Fehlerantwort (normiert nach prEN 18222:2025)

Generiert durch `APIUtilsDPP.create_generic_response()`:

```
{
  "message": [
    {
      "code": 404,
      "messageType": "ERROR",
      "correlationId": "ClientErrorResourceNotFound",
      "text": "DPP not found",
      "timestamp": "2026-05-08T10:00:00.000Z"
    }
  ]
}
```

Fehlerbehandlungsstrategie im Controller

- **Validierungsfehler** (z. B. fehlendes Pflichtfeld): `400 Bad Request`
- **Nicht gefunden**: `404 Not Found`

- **AAS-API-Fehler:** Werden in den meisten Methoden geloggt (`logger.warn`) und führen zu einer Fallback-Antwort, meist `500`
- **Unerwartete Exceptions:** Immer `500 Internal Server Error` mit `catch (Exception e)` – alle Exceptions werden aufgefangen

8. Bekannte Einschränkungen & Verbesserungspotenzial

#	Problem	Betroffene Stelle	Empfehlung
1	Doppelter Code <code>extractAndFilterSubmodels</code>	<code>APIController</code> + <code>APIUtilsDPP</code>	Konsolidieren, einheitliche Whitelist
2	<code>PATCH /elements/{path}</code> adressiert <code>dpps.0.</code> (immer erstes Array-Element)	<code>updateElement()</code>	MongoDB <code>arrayFilters</code> verwenden
3	<code>collectAssetInformation</code> und <code>collectAdministration</code> machen identische HTTP-Calls	<code>APIUtilsDPP</code>	In eine Methode zusammenfassen
4	Kein atomares Update in <code>PATCH</code> <code>/dpps/{dppId}</code>	<code>updateDpp()</code>	MongoDB-Transaktionen nutzen
5	<code>isBase64Encoded()</code> kann False Positives liefern	<code>Base64DPP</code>	Explizites Präfix oder Längencheck
6	<code>aasRegistryTemplate</code> wird im Konstruktor mit potenziell <code>null</code> -URI erstellt	<code>APIController</code>	Null-Check vor <code>MongoClients.create()</code>
7	Antwortformat bei <code>readDppById</code> doppelt verschachtelt (<code>assetInformation.assetInformation</code>)	<code>readDppById()</code> , <code>readDppByProductId()</code>	Response-Mapping korrigieren
8	<code>readElement</code> castet intern über (<code>POJONode</code>)	<code>readElement()</code>	Direkt <code>APIUtilsDPP.getDppById()</code> nutzen
9	Keine Sortierung bei <code>readDppByProductId</code>	<code>readDppByProductId()</code>	<code>Aggregation.sort(Sort.by("createdAt").descending())</code> hinzufügen
10	// TODO: mach den payload richtig!!! (Originalkommentar)	<code>readDppById()</code>	Payload-Struktur finalisieren

Hinweis: Dieses Dokument wurde mithilfe von Künstlicher Intelligenz (Claude Opus 4.7) erstellt.